

Feature Preparation Report for SLURRY_FREE_ACID Prediction

TSP Fertilizer Process Analytics Project

July 16, 2026

1 Purpose

This report summarizes the preparation workflow executed in the notebook `02_feature_engineering_data_preparation`. The goal was to build a leakage-safe, time-aware machine-learning dataset for predicting `SLURRY_FREE_ACID` one minute ahead. The preparation stage is not a modeling stage; it defines the input matrix, the target framing, and the train/validation/test structure that future models will use.

The supervised target used here is:

$$Y_{t+1} = \text{SLURRY_FREE_ACID}_{t+1}$$

and the model input is constructed from measurements available at time t or earlier.

2 What the executed notebook produced

The notebook run produced the following concrete outputs.

Metric	Value
Rows loaded from <code>tsp_1min.csv</code>	44,640
Original sensor features (excluding target)	21
Chronological rows after target alignment	44,640
Training rows	31,031
Validation rows	6,650
Test rows	6,650
Features retained after filtering	346

Table 1: Key dataset and split statistics from the executed notebook.

3 Preparation decisions and their modeling meaning

3.1 1. Leakage-safe target framing

The notebook explicitly framed the task as a one-step-ahead regression problem. The target was shifted by one minute into the future so that the model cannot use the future acid value during training or inference.

What was produced. The notebook created the future target column `SLURRY_FREE_ACID_t_plus_1min`. The final supervised table contains the time index, engineered features, and this future target.

Why it matters for modeling. This is a critical design decision because it makes the setup realistic for deployment. A model trained on this table is not allowed to use the future target value, so its predictions reflect what would have been possible at the time of prediction. This prevents an optimistic evaluation that would otherwise arise from target leakage.

3.2 2. Chronological splitting instead of random splitting

The data was split in chronological order rather than shuffled. The train period covered the earliest part of the dataset, followed by validation and then test periods in time sequence.

Split	Rows	Start	End
Train	31,031	2026-01-01 00:00 UTC	2026-01-22 17:08 UTC
Validation	6,650	2026-01-22 17:09 UTC	2026-01-27 08:54 UTC
Test	6,650	2026-01-27 08:55 UTC	2026-01-31 23:58 UTC

Table 2: Chronological train/validation/test split produced by the notebook.

What it produced. The notebook produced three disjoint time segments with no overlap and no shuffling.

Why it matters for modeling. This avoids a common pitfall in time-series industrial data: using future operating states that are nearly identical to the ones being evaluated. A chronological split makes the evaluation more faithful to real deployment conditions and discourages over-optimistic scores.

3.3 3. Feature engineering from current and past values only

The feature engineering layer generated multiple families of features from the original sensors:

- original sensor values at time t ;
- lags at 1, 2, 5, 10, 30, and 60 minutes;
- rolling means, medians, standard deviations, minima, and maxima over 5, 10, 30, and 60 minutes;
- exponential moving averages;
- differences and percentage changes;
- rolling slopes;
- time-of-day and shift-based cyclic features;
- process ratios and interaction terms.

What it produced. The notebook built a large engineered feature matrix from the original 21 sensors. The retained documentation table contains 346 features after filtering.

Why it matters for modeling. This step gives the model a richer description of process behavior, including delayed responses, recent trends, thermal balance, and operating regime effects. For industrial data, these features can be more informative than raw measurements alone because the target often depends on short-horizon dynamics rather than a single point measurement.

3.4 4. Missing-value treatment strategy

The notebook treated missing values using a train-only median imputation strategy. Missing values were not backward-filled, because that would use future observations.

What it produced. The notebook generated a missing-value treatment table for the engineered features, describing the source of missingness and the planned imputation rule. The preprocessing pipelines were fit on the training split only.

Why it matters for modeling. This avoids leakage from the future and makes the preprocessing procedure reproducible. Median imputation is a reasonable baseline in industrial settings because it is robust to spikes and preserves a stable center. The strategy is especially important for lagged and rolling features, which naturally have warm-up gaps and missing values at the beginning of the series.

3.5 5. Feature filtering before scaling

Before scaling, the notebook applied three deterministic filters using only the training split:

- all-null columns were removed;
- low-variance columns were removed;
- highly correlated features were removed using a correlation threshold of 0.98.

Filter	Removed count
All-null training columns	0
Low-variance training columns	0
High-correlation columns above 0.98	453
Remaining features	346

Table 3: Feature filtering results from the executed notebook.

What it produced. The feature matrix was reduced from 799 engineered features before filtering to 346 retained features after filtering.

Why it matters for modeling. The filtering step removes useless or redundant variables before model training. In this run, the dominant issue was redundancy rather than nulls or nearly constant columns. This is a useful result because correlated features can destabilize regularized models and make tree-based models less interpretable. The remaining 346 features still provide a broad input space, but with much less duplication.

3.6 6. Scaling variants for different model families

The notebook created multiple preprocessed versions of the same feature matrix:

- unscaled features for tree models;
- robust-scaled features for outlier-aware linear and neural models;
- standard-scaled features for PCA, regularized models, and diagnostics;
- MinMax-scaled features for bounded-input neural-network experiments.

What it produced. The prepared datasets were named `X_train_tree`, `X_valid_tree`, `X_test_tree`, and the corresponding scaled versions for robust, standard, and MinMax preprocessing. Each dataset has 346 columns.

Dataset	Rows	Columns
<code>X_train_tree</code>	31,031	346
<code>X_valid_tree</code>	6,650	346
<code>X_test_tree</code>	6,650	346
<code>y_train</code>	31,031	1
<code>y_valid</code>	6,650	1
<code>y_test</code>	6,650	1

Table 4: Prepared machine-learning dataset shapes.

Why it matters for modeling. Different model families require different input scaling. Tree-based models do not benefit from scaling, while linear models, PCA, distance-based methods, and neural nets often do. Preparing multiple versions makes the next modeling stage more flexible and reduces the need for ad hoc preprocessing later.

3.7 7. Feature-selection diagnostics

The notebook also computed a mutual-information ranking against the future target. This was done without fitting a predictive model, so it provides a model-agnostic diagnostic view of nonlinear dependency.

What it produced. The top-ranked features from the mutual-information table include:

- `FLOW_RATE_PHOSPHORIC_ACID_2_roll_60min_min`
- `PRESSURE_VAPEUR_roll_60min_min`
- `FLOW_RATE_GROUND_PHOSPHATE_roll_60min_max`
- `DEPRESSURE_SECHEUR_roll_60min_min`
- `TEMPERATURE_VAPEUR_roll_60min_max`

Why it matters for modeling. These results are useful as a feature-selection diagnostic. They indicate that recent rolling behavior and process-level balance variables are likely to be important for downstream models. However, mutual information is only a diagnostic; the next modeling stage should validate these rankings experimentally with actual fitted models.

4 Implications for the modeling stage

The preparation stage produced a dataset that is ready for modeling, but it also reveals several modeling expectations.

- The task is now clearly framed as a one-minute-ahead regression problem.
- The data is leakage-safe, chronologically split, and suitable for realistic evaluation.

- The feature space is broad and process-informed, but compressed from 799 candidates to 346 retained variables.
- The current preparation favors robust, interpretable baselines while still allowing more complex families such as gradient boosting, regularized linear models, and neural architectures.
- Because the target is likely affected by rare operational excursions, models should be evaluated not only by average error, but also by how they behave during abnormal regimes.

5 Practical recommendation

The most appropriate next step is to benchmark a small set of model families on the prepared datasets:

1. tree-based models on the unscaled data;
2. regularized linear models on the standard or robust-scaled data;
3. gradient boosting models on the unscaled data;
4. a simple neural-network baseline on the robust or MinMax-scaled data.

This sequence is sensible because the preprocessing stage has already made the feature matrix consistent, leakage-safe, and dimensionally reduced. The modeling notebook can now focus on comparing predictive performance rather than re-engineering the data foundation.